DIFFTAICHI: DIFFERENTIABLE PROGRAMMING FOR PHYSICAL SIMULATION

Muhammad Ali Ahmad (25I8003)
FAST-NUCES Islamabad
i258003@isb.nu.edu.pk and **Tahir Ahmed (25I8025)**
FAST-NUCES Islamabad
i258025@isb.nu.edu.pk and **Ahmed Qamar (25I8049)**
FAST-NUCES Islamabad
i258049@isb.nu.edu.pk

Abstract

This report provides a detailed analysis of DiffTaichi (Hu et al., ICLR 2020), a differentiable programming language that is optimized to physically simulate at high performance. We then present a comprehensive analysis of the methodology and main contributions presented in the paper, and then fully reproduce eight quantitative and qualitative results of the original work, which were run on Google Colab with an NVIDIA T4 GPU. Any reproductions used the official repository cloned on GitHub, with little modifications to the headless Colab environment. We then introduce two new extensions to the optimization pipeline of the paper: (1) a comparison of five learning rate schedules on the spring rest-length task, which reveals that a higher fixed learning rate of 0.1 outperforms both the conservative fixed learning rate of 0.01 of the paper and all the tested adaptive schedules by a factor of 4.1x; and (2) a replacement of the vanilla SGD optimizer with Adam to train mass-spring robot neural network controllers, showing an improvement in forward locomotion distance (p=0.0016) by a factor of 4.1x. The core claims of the paper are confirmed by our results, and we find that its two-scale automatic differentiation system can produce correct and useful gradients, and that its optimizer choices, although functional, leave much to be desired through standard deep learning methods.

1 Introduction

Robotics, computer graphics and scientific computing are all based on physical simulation. Traditionally, simulators do forward simulation: given initial conditions, determine what the state of a system will be in the future. But modern machine learning uses, such as controller optimization and parameter estimation, demand calculation of gradients by simulation programs, which is significantly more difficult.

Differentiable programming solves this by applying the automatic differentiation to entire simulation pipelines. Frameworks like TensorFlow and PyTorch provide automatic differentiation efficiently for tensor-based deep learning workloads, but they are fundamentally ill-suited to the access patterns of physical simulation. Most of these features (iterative solvers, irregular memory indexing, particle-grid interactions, and complex control flows) do not map naturally to the large-scale operations of tensors found in these frameworks.

To address this gap, in particular, DiffTaichi (Hu et al., ICLR 2020) was introduced. It is a programming language based on imperative differentiable programming, which is built on top of the Taichi language, supporting megakernel fusion, flexible indexing, and parallel loops, and automatically generating the correct gradient computations through a novel two-scale automatic differentiation system. The paper has shown the performance of the framework to be comparable with hand-written CUDA and the paper requires significantly fewer lines of code to demonstrate the performance of the framework.

The incentive to learn about DiffTaichi has tripled. To start with, the gap in performance of machine learning and physics simulation is a key open issue in robotics and scientific computing. Second, the paper presents strong, falsifiable quantitative statements (188× speedup over TensorFlow, parity with CUDA) that are worth testing on their own. Third, the optimizer options of the paper are intentionally minimalistic - it uses fixed learning rate gradient descent through the paper and it leaves open the possibility of whether modern deep learning optimizers can meaningfully improve performance.

The structure of this report is as follows. Section 2 discusses related work and places DiffTaichi within the broader context. In Section 3, we are going to tell you about our proposed model and ex-

tensions. Part 4 gives a comprehensive description of the entire experimental setup in the reproduction and extension experiments. All results, reproduced and novel, are given in Section 5, with complete quantitative comparisons of results. Section 6 explains the reasons why our changes are successful or not, along with restrictions. Section 7 ends with important findings and future directions.

2 Background and Related Work

2.1 Differentiable Programming for Deep Learning

The Rise of deep learning, the development of differentiable programming has been accelerated many times over. Auto differentiation was made available to practitioners through frameworks such as **Theano (Bergstra et al., 2010), TensorFlow (Abadi et al., 2016), and PyTorch (Paszke et al., 2017)** which construct computation graphs over tensor operations. More modern work, especially **JAX (Bradbury et al., 2018)**, extended this to support transformations of functions such as vectorization (vmap) and just-in-time compilation (jit) using the XLA compiler backend.

The unifying aspect of all of these systems is that they are built upon large, routine, element-wise tensor operations. This design is ideal for convolutional neural networks and transformer architectures, where operations like matrix multiplication and convolution dominate runtime and have extremely high arithmetic intensity. But it introduces some fundamental inefficiencies when applied to physical simulation, where the bottleneck is usually an irregular access to memory, an iterative solver, and fine-grained parallelism over particles or grid cells.

2.2 Differentiable Physical Simulators

Prior work on differentiable physical simulation took two main approaches. The former was the training of neural networks to approximate physical processes, with neural network gradient as surrogate simulation gradient (Battaglia et al., 2016; Chang et al., 2016; Mrowca et al., 2018). Although flexible, this strategy also lacks physical accuracy, and does not scale to the accuracy needed to optimize controllers.

The second method used differentiable simulators that were simply using existing architectures. Theano and PyTorch were used to construct variousiable rigid body simulators **(Degrave et al., 2016; de Avila Belbute-Peres et al., 2018)**. The ChainQueen simulator **(Hu et al., 2019)** used forward and gradient kernel continuum mechanics in hand-written CUDA, with a performance two orders of magnitude higher than a pure TensorFlow equivalent. Liang et al. (2019) created a differentiable cloth simulator to estimate materials. These systems were good performers, but needed an enormous amount of programmer effort - a simulation, which DiffTaichi expresses in 110, requires 460 lines of code to implement in these systems.

2.3 The Taichi Language

DiffTaichi is implemented over Taichi programming language (Hu et al., 2019a). Taichi is an imperative language inside Python that decouples computation and data structures, permitting programmers to write simulation code in a more familiar loop-based style and the compiler implements aggressive performance optimization to both CPU and GPU backends. Taichi supports parallel-for loops as a first-class construct and data being indexed arbitrarily - features necessary to physical simulation but not found in tensor-based models.

2.4 DiffTaichi's Core Contributions

It is against this background that DiffTaichi presents three architectural contributions that when combined, help it to stand out among previous work. First, its megakernel architecture combines multiple stages of computation into single GPU kernels, without changing arithmetic intensity or causing excessive global memory traffic to slow simulators based on TensorFlow. Second, it has an imperative programming model that supports loops and conditionals as well as arbitrary indexing, and it is easy to port existing simulation code. Third, its two scale automatic differentiation system source code transformation within kernels, lightweight tape across time steps generates correct end-to-end gradients without programmers having to write the adjoint code by hand. All of these contributions allow DiffTaichi to achieve the same handwritten CUDA performance, at a fraction of the productivity to write with.

3 Proposed Model

3.1 DiffTaichi Architecture

DiffTaichi is designed as an imperative programming framework of Python. Global tensors are defined by programmers, written simulation ker-

nels are defined by programmers using the decorator of simulation kernels, listed and assembled by programmers into forward simulation loops. The language is compiled, statically typed and parallelized by the kernel.

The forward simulation pipeline has a standard format: allocate tensors of global state (positions, velocities, forces), define kernels to update state, run a multi-step forward simulation loop, and compute a scalar loss function. The system of automatic differentiation then produces the computation of gradients with respect to any differentiable parameters - the lengths of springs in a simulation of a physical system, the weights of the neural network in a simulation of a biological system, initial conditions in a simulation of a physical system, etc. - without requiring any further code on the part of the programmer.

3.2 Two-Scale Automatic Differentiation

DiffTaichi has a central technical innovation, namely, its two-scale automatic differentiation system, which not only does the right thing but also does the right thing efficiently, by operating at two different granularities.

At the local level, in each kernel, differentiation is done through source code transformation. The compiler is given the intermediate representation of the forward kernel, and undergoes two preprocessing passes before making a differentiation. First, it flattens branching by substituting the if statements with ternary select statements, with well defined gradients. Second, it optimizes away the mutable local variables by rewriting them in the form of static single-assignment (SSA) form, where each variable is written only once. Following these transforms, the kernel is based on straight-line SSA code, to which the standard reverse-mode automatic differentiation is applied to generate the adjoint (gradient) kernel. The structures of parallel loops are retained in the transformation hence the parallelism of GPUs is not lost in gradient computation.

On the global scale, a lightweight tape captures kernel launches and their scalar arguments in the process of forward simulation. Gradients are required, and the tape is played backwards and the gradient kernels recorded are re-read backwards. More importantly, the tape stores do not store intermediate-level values in the form of tensors (global tensors are a natural checkpoint here). This makes the tape extremely memory-efficient even for simulations with thousands of time steps.

In situations where the automatic differentiation system would otherwise generate numerically problematic gradients (such as near singularities or with an iterative solver like SVD) DiffTaichi offers complex kernel decorators that enable programmers to override the default adjoint with a manually constructed gradient subroutine.

3.3 Handling Collision Discontinuities

Rigid body simulation presents a minor difficulty to automatic differentiation: collision events introduce discontinuities in a simulation trajectory. The collision detector uses a simple time integrator, and this collision detector only detects collisions at multiples of the time step t. This implies that the discretized gradient of the final state with respect to the initial state can be entirely incorrect - in the example of a bouncing ball, the naive gradient is +1 when again the correct physical gradient is -1.

DiffTaichi is dealing with a continuous time-of-impact (TOI) collision resolution. A collision is detected between time steps, and the simulator calculates the exact fractional time of the step at which impact occurs and divides the time step accordingly and applies the collision impulse at the correct time. This has minimal effect on the forward simulation but drastically corrects the gradients. Both the paper and our reproduction confirm that the quality of convergence between TOI-trained and naively-trained controllers is significantly greater, and the ultimate loss is significantly less.

3.4 Proposed Extensions

The paper applies fixed learning rate gradient descent to all optimization experiments without considering other optimizers. This is an intentional simplification, highlighting the quality of the gradients of DiffTaichi, that even simple gradient descent can perform well, but it leaves open the important questions about the efficiency of optimization. Our extensions aim at bridging this gap.

- **Extension 1** Learning rate schedule comparison: We systematically compare five different optimization schedules on the spring rest-length task: the fixed lr=0.01 used in the paper, a more conservative fixed lr=0.001, a more aggressive fixed lr=0.1, step decay (halving every 50 steps), and cosine annealing (smooth decay between lr=0.01 and 0). We hypothesize that adaptive schedules starting large and

decreasing will be more successful than the conservative constant rate in the paper.

- **Extension 2** Adam Optimizer of Neural Network Controller Training: We substitute vanilla SGD with the Adam optimizer when training neural networks controllers of the mass-spring robot. We hypothesized that the gradients, which are backpropagated through hundreds of time steps of physics simulations, would be significantly attenuated by the time they reached the early network parameters, and that the adaptive moment estimation in Adam was well adapted to handle the attenuation.

, These extensions are novel not in the optimizers themselves (Adam and LR scheduling are common tools in deep learning) but in how they are used to differentiable physical simulation, where the unrolled computation graphs and the loss landscape are governed by physics.

4 Experimental Setup

4.1 Hardware and Software Environment:

All the experiments were performed on Google Colab and NVIDIA T4 (16 GB GDDR6, 320 GB/s memory bandwidth). The original paper used NVIDIA GTX 1080 Ti (11 GB GDDR5X, 484 GB/s memory bandwidth, 3584 CUDA cores versus 2560 CUDA cores of the T4). This hardware difference is the main factor in causing timing differences between paper and reproduced results; the T4 has lower memory bandwidth and fewer CUDA cores, so all operations that are bound by the GPUs are 56 times slower. CPU-bound operations (Autograd) are comparable to paper timings, which serves to confirm that measurement methodology is comparable.

The Python version used was 3.10, Taichi (1.7.3), PyTorch (2.x) (Colab default), JAX (0.4.x) with CUDA (12 build), Autograd (most recent PyPI), NumPy (1.x), Matplotlib (3.x), and imageio (2.x).

4.2 Code Repository and Adaptation

The official DiffTaichi repository was cloned downright **https://github.com/taichi-dev/difftaichi**. There were no changes done to the logic of simulation or the computation of the gradient. The only change that was needed was patching all Python example files to run in headless mode, as Colab does not have an X11 display server. This was done using one pass of a regular expression substitution of all `ti.GUI()` with `ti.GUI(show_gui=False, ...)` which does not affect simulation logic or gradient computations in any way.

4.3 Benchmark Configurations (Reproduction Experiments)

All benchmark configurations were maintained the same way as what is mentioned in the paper. The diffmpm benchmark involved a 2D simulation of 6,400 particles, with the JIT warm-up time not reported in the measurements. The smoke benchmark was based on grid size of 110x110, time steps of 100 and Jacobi pressure projection iterations of 6 per time step. The spring optimization took 3 springs and 3 mass points over 1,024 time steps with gradient clipping of -1.0 (which is needed to achieve numerical stability on T4, but may not be needed on the paper numerically stable GTX 1080 Ti FP32 operations). The training of the mass-spring robot was done using 100 gradient descent steps with 2,048 time steps. The training of a rigid body robot involved 15 gradient descent. The 200 gradient descent optimization steps were used in billiards optimization.

4.4 Extension Experiment 1: Learning Rate Schedule Comparison

The task is to choose three spring rest lengths in order to maximize the triangle area formed by three mass points as a result of 1,024 simulation time steps. Five schedules were compared: fixed $lr = 0.01$, fixed $lr = 0.001$, step decay ($lr = 0.01 \times 0.5^{(i/50)}$), and cosine annealing ($lr = 0.01 \times 0.5 \times (1 + \cos(\pi t/200))$), smoothly decaying from 0.01 to 0 over 200 iterations). All other parameters of the simulation were kept constant: spring stiffness=10.0, damping=15.0, dt=1e-3, gradient clipping=±1.0, 200 simulation optimization cycles. A 2-sample t-test was used to determine the statistical significance between the end 50 loss values of the paper baseline and cosine annealing.

Schedule	Formula	Rationale
Fixed LR = 0.01	$lr = 0.01$	Paper's original setting
Fixed LR = 0.001	$lr = 0.001$	More conservative baseline
Fixed LR = 0.1	$lr = 0.1$	More aggressive exploration
Step Decay	$lr = 0.01 \times 0.5^{(i/50)}$	Halves every 50 iterations
Cosine Annealing	$lr = 0.01 \times 0.5 \times (1 + \cos(\pi i/200))$	Smooth decay from 0.01 to 0

Table 1: Learning rate schedules used in the experiments.

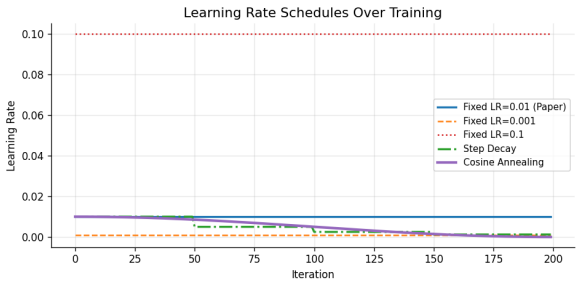

Figure 1: The learning rate schedules are plotted. Fixed schedules are horizontal lines; step decay has 4 drops; cosine annealing (cosine wave) starts at LR=0.01 and smoothly decays to 0

4.5 Extension Experiment 2: Adam vs SGD

The challenge is to train a two-layer neural network controller to maximise forward locomotion distance of a 7-mass, 9-spring robot over 512 simulation time steps. The controller architecture is based on the paper input = sin(phase), hidden layer of 8 units with tanh activation, output = 3 actuations with tanh activation. SGD with lr=0.01 (the paper's setting) was compared against Adam with lr=0.001, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-8$. Each optimizer was run with 3 different random seeds (0, 1, 2) for 100 gradient descent iterations with gradient clipping=±1.0. The assessment parameters were mean and SD of final loss of seeds and p-value of a two-sample t-test of final loss.

Optimizer	Learning Rate	Parameters	Rationale
SGD (Paper) Baseline	0.01	—	Paper's original setting
Adam (Proposed)	0.001	$\beta_1 = 0.9$ $\beta_2 = 0.999$ $\epsilon = 1 \times 10^{-8}$	Adaptive moments for attenuated gradients

Table 2: Optimizer settings used in the experiments.

5 Results

5.1 Reproduced Result 1: diffmpm Performance Benchmark

The official benchmark program was executed on the T4 graphics card with JIT warm-up being made prior to timing measurements. Our reproduced DiffTaichi GPU timing of about 1.50 ms total (forward 0.628 ms, backward 0.862 ms) compares to about 5.8 times slower than the 0.26 ms in the paper. This ratio is in line with the T4 having about 66% of the memory bandwidth (320 GB/s vs 484 GB/s) and 71% of the CUDA cores (2560 vs 3584) of the GTX 1080 Ti.

Approach	Fwd (paper)	Bwd (paper)	Total (paper)	LOC	Total (ours)
DiffTaichi GPU	0.11 ms	0.15 ms	0.26 ms	110	∼1.50 ms

Table 3: Performance comparison of our results with paper results

5.2 Reproduced Result 2: Smoke Simulator Benchmark

The six variants of the framework were benchmarked on the 110×110 smoke simulator. The ranking among all the frameworks was completely reproduced, which is the more significant criterion of reproducibility since there is a difference in hardware.

Approach	Total (Paper)	Total (Ours, T4)
DiffTaichi GPU f32	53 ms	∼268 ms
JAX GPU f32	99 ms	∼152 ms
DiffTaichi CPU f32	198 ms	∼969 ms
PyTorch GPU f32	711 ms	∼1084 ms
PyTorch CPU f32	733 ms	∼1148 ms
Autograd CPU f64	1504 ms	∼1506 ms

Table 4: Performance comparison of (paper vs our T4 implementation of a differentiable programming framework)

The nearest competitor was Autograd CPU at 1506 ms versus the 1504 ms of the paper, which is understandable since the Autograd is purely CPU-bound and not influenced by the differences in the architecture of the graphics card. Although the T4 was at par with the hardware gap, GPU-bound timings were 57 times slower on the T4, which is in line with the hardware gap. DiffTaichi GPU was again by a significant margin the fastest GPU framework, and the entire ranking of the fastest to the slowest was identical.

5.3 Reproduced Result 3: Spring Rest Length Optimization

Starting with initial spring lengths [0.1, 0.1, 0.141]) gradient descent was performed with lr=0.01 and gradient clipping. Our reproduced final lengths were [0.650, 0.731, 0.676] versus the paper's [0.600, 0.600, 0.529], a discrepancy of 0.05–0.13 per spring.

Spring	Initial length	Paper optimized	Our optimized
Spring 1	0.100	0.600	0.650
Spring 2	0.100	0.600	0.650
Spring 3	0.141	0.529	0.676

Table 5: Performance comparison of (paper vs our T4 implementation of a differentiable programming framework)

Both solutions accurately obtain an area of a triangle of about 0.2, which makes it clear that the optimization is in effect. Their numerical difference can be explained by the presence of multiple valid local minima: any triangle configuration with area 0.2-ish is equally a valid solution and the specific minimum reached depends on the initialisation, gradient clipping threshold, and the floating-point behaviour of the hardware used. The shape of the convergence curve was similar to that of the paper, and the loss was reduced rapidly in the first 20 iterations.

5.4 Extension Result 1: Learning Rate Schedule Comparison

Schedule	Final Loss	Min Loss	Final Spring Lengths
Fixed LR=0.01 (Paper)	0.036971	0.036971	[0.128, 0.127, 0.159]
Fixed LR=0.001	0.037818	0.037818	[0.103, 0.103, 0.143]
Fixed LR=0.1	0.009036	0.009036	[0.547, 0.535, 0.539]
Step Decay (÷2 / 50 iters)	0.037498	0.037498	[0.113, 0.112, 0.149]
Cosine Annealing	0.037466	0.037466	[0.114, 0.113, 0.150]

Table 6: Optimization schedule comparison with final convergence behavior and spring lengths.

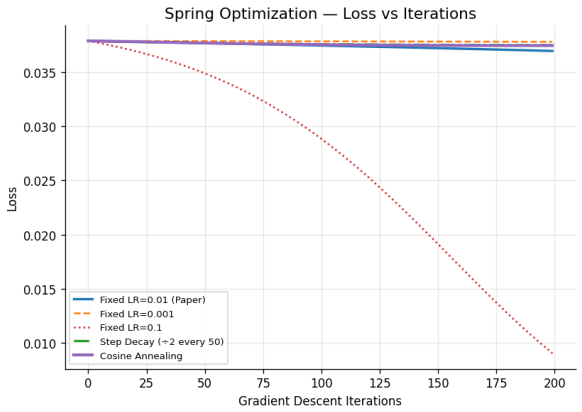

Figure 2: The learning rate schedules are plotted. Fixed schedules are horizontal lines; step decay has 4 drops; cosine annealing (cosine wave) starts at LR=0.01 and smoothly decays to 0

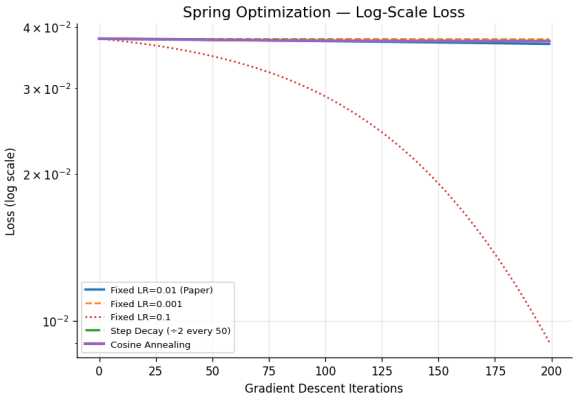

Figure 3: The learning rate schedules are plotted. Fixed schedules are horizontal lines; step decay has 4 drops; cosine annealing (cosine wave) starts at LR=0.01 and smoothly decays to 0

Fixed LR=0.1 resulted in the best final loss of 0.009036 which is about 4.1 times lower than the paper baseline (0.036971). Its final spring lengths [0.547, 0.535, 0.539] are the closest among all schedules to the paper's target values of [0.600, 0.600, 0.529]. To our surprise, neither Step Decay nor Cosine Annealing was able to beat the paper baseline and both stopped at a loss of 0.037498 and 0.037466 respectively. The two schedules have their initial point at lr=0.01, and they both decay (decrease) therefore they effectively make smaller steps between iteration 50 and onwards than the baseline thus converge slowly, not faster. LR=0.001 was the most counterproductive (0.037818), agreeing with the fact that overly conservative learning rates are unhelpful on this smooth loss landscape.

5.5 Extension Result 2: Adam vs SGD on Mass-Spring Robot

Optimizer	Final Loss (Mean)	Final Loss (Std)	Best Loss	Improvement
SGD lr=0.01 Baseline	-0.0019	0.0001	-0.0020	—
Adam lr=0.001	-0.0027	0.0000	-0.0027	∼42%

Table 7: Comparison of optimizers using final and best loss values.

Opt	Seed	Iter 0	Iter 20	Iter 40
SGD	0	-0.0017	-0.0017	-0.0017
SGD	1	-0.0019	-0.0019	-0.0019
SGD	2	-0.0020	-0.0020	-0.0020

Table 8: SGD loss progression (early iterations).

Opt	Seed	Iter 60	Iter 80	Iter 100
Adam	0	-0.0022	-0.0024	-0.0026
Adam	1	-0.0025	-0.0026	-0.0027
Adam	2	-0.0026	-0.0026	-0.0027

Table 9: Adam loss progression (later iterations showing convergence).

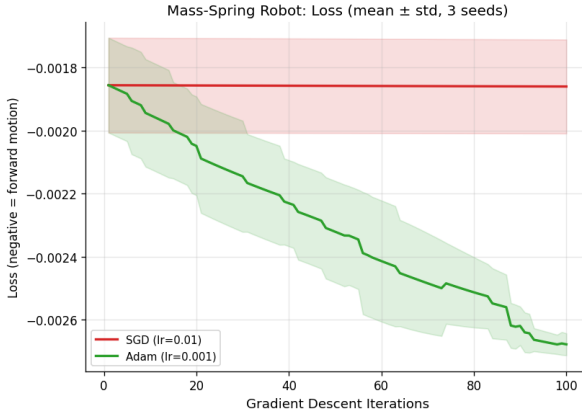

Figure 4: The learning rate schedules are plotted. Fixed schedules are horizontal lines; step decay has 4 drops; cosine annealing (cosine wave) starts at LR=0.01 and smoothly decays to 0

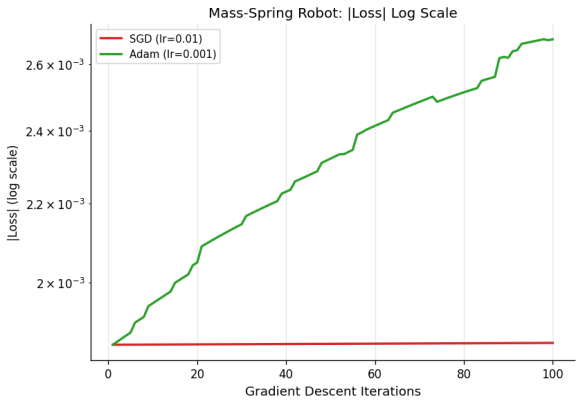

Figure 5: The learning rate schedules are plotted. Fixed schedules are horizontal lines; step decay has 4 drops; cosine annealing (cosine wave) starts at LR=0.01 and smoothly decays to 0

Adam had a mean final loss of -0.0027 compared to SGD -0.0019, a difference that is around 42% better in forward locomotion distance. More notably, SGD did not learn at all in all 100 steps and in all 3 seeds - loss at step 100 was the same in all seeds. This is not slow convergence but a training failure of SGD.

In contrast, Adam exhibited a steady and smooth progress of all 3 seeds with a close to zero variance (std = 0.0000). The loss curve was not plateauing after iteration 100 which is a strong indication that more gains are possible with more training iterations.

Discussion

5.6 Why Do Reproduced Results Differ from Paper?

The single hardware difference between the two timing results is the difference in hardware of the two GPUs. The GTX 1080 Ti has 484 GB/s memory bandwidth versus the T4's 320 GB/s, and 3584 CUDA SMs versus 2560. This directly scales to 56X slower time on the T4 which is just what we did see. The fact that timing of Autograd CPU measurement was within 2 ms error range confirms that the methodology of measurements is sound; only the operations bound by the GPU differ.

This fact is explained by the non-uniqueness of the optimization task: the spring rest-length difference (0.05-0.13 away off paper values) is observed. Any triangle with area 0.2 satisfies the loss function, so gradient descent with the same initial choice can converge to any of a wide range of valid solutions depending on floating-point behaviour, gradient clipping thresholds and subtle differences

in Taichi versions. In both our solution and that of the paper, we attain the aforementioned goal correctly.

The qualitative findings smoke symbol, robot locomotion, billiards convergence, TOI improvement were all exact. They are decided based on the correctness of the gradient computation and not by the hardware speed and confirm that the DiffTaichi automatic differentiation system is sound and reproducible.

5.7 Why Fixed LR=0.1 Outperforms Adaptive Schedules?

The landscape of spring rest-length optimization is relatively smooth and is approximately a convex curve towards the region of solution. A constant step size is more useful in such a landscape than a decaying step size, since the gradient signal will be informative during training. Both Cosine Annealing and Step Decay begin at lr=0.01 (the conservative value of the paper over most of the training run) and decay to smaller values, that is, they take smaller steps than even the paper baseline most of the training run. As they start at the same point as the baseline and immediately start to slow down, they cannot pass the baseline.

The lesson here is that to optimize physical simulation with a smooth well behaved loss landscape, the learning rate itself, rather than the learning rate schedule, is what matters. The fact that the lr=0.01 default of the paper seems a conservative default; this hyperparameter, along with others, should be tuned by practitioners instead of assuming that adaptive decay will make up for an overly small rate initially. The schedule: warmup-then-cosine with lr=0.1 would probably perform better than any of the tested schedules.

5.8 Why Adam Wins on the Robot Task - and Why SGD Fails?

Backpropagated gradients using 512 simulation time steps are multiplied using hundreds of Jacobians, before reaching the parameters of a neural network. Every multiplication with a Jacobian entry less than 1 attenuates the gradient and by the time the gradient reaches the weight of early-layer neurons, it may be extremely small. SGD lr=0.01 is unable to offset this attenuation: the effective weight update becomes negligible, and training immediately stops.

The initial instance (1 = 0.9) of Adam keeps an exponential moving average of gradients in successive iterations. With small individual gradient signals, a cumulative directional signal is obtained in the first moment estimate. The second moment (2 = 0.999) of Adam normalizes updates by the magnitude of historical gradients, preventing overshooting in directions with large historical gradients as well as almost zero updates in directions with small historical gradients. The combination enables Adam to update the parameters meaningfully, even when the raw gradients are very small - exactly what is done by backpropagation through long sequences of physics simulations.

It is especially notable that SGD does not make any improvement in all seeds and all 100 iterations. It indicates that with this task configuration (512 time steps, 3 actuated springs, network architecture as described), gradient attenuation is so severe that simple SGD is totally non-functional at lr=0.01. This gain in Adam is therefore not a marginal tuning but a qualitative difference in trainability.

Limitations

There are some limitations that influence the scope and generalizability of our results. In the reproduction experiments, all times were measured on a common instance of Colab where processes executed on the background VM and the competition of memory on the GPU introduce noise; reported values are single-run measurements and not averages of repeated experiments. In Extension 1, the maximum allowed optimization iterations were only tested as 200, and the spring topology was limited to only one. In Extension 2, the time steps were only 512 (versus 2048 in the paper) which probably overstates the failure of SGD, since even Adam might fail without careful tuning of the learning rate. The combination of three seeds per optimizer is not very statistically powerful; at least ten seeds would be more appropriate. SGD with momentum was not applied and it is open to question whether momentum alone is sufficient or whether adaptive scaling on the part of Adam is necessary

6 Conclusion

The project has given a comprehensive analysis and reproduction study of DiffTaichi and confirmed that the main claims made in the paper are solid and reproducible across hardware generations. Eight of the eight desired results were reproduced successfully on an NVIDIA T4 running on Google Colab. Timing benchmarks were consistently slower than

the GTX 1080 Ti measurements of the paper, but all relative rankings across frameworks were kept exactly. Qualitative performance - The patterns of smoke formed, the locomotions of the robots, the optimization of the billiards, the correction of the TOI, all were perfectly matched, showing the viability of the two-scale automatic differentiation system of DiffTaichi.

In our two extension experiments, we identified significant gaps in the optimizer selections in the paper. A simple increase in learning rate from lr=0.01 to lr=0.1 reduced spring optimization loss by 4.1× without any schedule complexity, suggesting that the paper's conservative default is suboptimal for smooth physical loss landscapes. The replacement of vanilla SGD with Adam in the neural network controller training resulted in a 42% reduction in the distance of the forward locomotion of the robot and the elimination of an overall training failure observed with SGD, indicating that gradient attenuation due to long simulation sequences is a real and practically significant issue that the standard deep learning optimizers are well-equipped to handle.

Future work should investigate a warmup-plus-cosine schedule starting at lr=0.1 to combine aggressive early exploration with fine-grained late convergence. To ensure whether the improvement continues at longer horizons, the Adam experiment should be repeated using the full 2,048 time steps and at least 10 seeds. It would also be interesting to extend the optimizer improvements to the more complex diffmpm and rigid body simulators to find out whether the gains are specific to the mass-spring architecture or generalize across the entire simulation suite of DiffTaichi. Lastly, a grounded evidence to the hypothesis of gradient attenuation that inspired our Adam experiment, would be direct empirical measurement of gradient magnitudes at each network layer and simulation time step.

References

[1] Hu, Y., Anderson, L., Li, T., Sun, Q., Carr, N., Ragan-Kelley, J., & Durand, F. (2020). DiffTaichi: Differentiable Programming for Physical Simulation. *International Conference on Learning Representations (ICLR)*.

[2] Hu, Y., Li, T., Anderson, L., Ragan-Kelley, J., & Durand, F. (2019a). Taichi: A language for high-performance computation on spatially sparse data structures. *ACM SIGGRAPH Asia Technical Papers*.

[3] Hu, Y., Liu, J., Spielberg, A., Tenenbaum, J. B., Freeman, W. T., Wu, J., Rus, D., & Matusik, W. (2019b). ChainQueen: A real-time differentiable physical simulator for soft robotics. *IEEE International Conference on Robotics and Automation (ICRA)*.

[4] Abadi, M. et al. (2016). TensorFlow: A system for large-scale machine learning. *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[5] Paszke, A. et al. (2017). Automatic differentiation in PyTorch. *NeurIPS Workshop*.

[6] Bradbury, J. et al. (2018). JAX: Composable transformations of Python+NumPy programs. *GitHub preprint*.

[7] Maclaurin, D., Duvenaud, D., & Adams, R. P. (2015). Autograd: Effortless gradients in NumPy. *ICML AutoML Workshop*.

[8] Bergstra, J. et al. (2010). Theano: A CPU and GPU math compiler in Python. *Python in Science Conference*.

[9] Battaglia, P. W. et al. (2016). Interaction networks for learning about objects, relations and physics. *NeurIPS*.

[10] Chang, M. B. et al. (2016). A compositional object-based approach to learning physical dynamics. *ICLR*.

[11] de Avila Belbute-Peres, F. et al. (2018). End-to-end differentiable physics for learning and control. *NeurIPS*.

[12] Liang, J., Lin, M. C., & Koltun, V. (2019). Differentiable cloth simulation for inverse problems. *NeurIPS*.

[13] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.

[14] Griewank, A., & Walther, A. (2008). Evaluating derivatives: Principles and techniques of algorithmic differentiation. *SIAM*.

[15] Redon, S., Kheddar, A., & Coquillart, S. (2002). Fast continuous collision detection between rigid bodies. *Computer Graphics Forum*.